

FastAPI API Security

Authentication, Authorization, JWT Tokenization & Refresh Tokens

A complete beginner-to-advanced reference for securing FastAPI APIs: password hashing, OAuth2, JWT access & refresh tokens, RBAC/scopes, token revocation, rate limiting, and production hardening.

Stack: FastAPI · PostgreSQL · SQLAlchemy · python-jose · passlib/argon2

Format: Commands-first, copy-pasteable, production-oriented

Table of Contents

1. Core Concepts — Authentication vs Authorization
2. Project Setup & Dependencies
3. Password Hashing (bcrypt / argon2)
4. Database Models (User table with PostgreSQL)
5. OAuth2 Password Flow in FastAPI
6. JWT Deep Dive — Structure, Claims, Signing
7. Issuing Access Tokens
8. Protecting Routes with Dependencies
9. Refresh Tokens — Theory & Rotation Strategy
10. Implementing Refresh Tokens (DB-backed, revocable)
11. Logout & Token Revocation / Blacklisting
12. Role-Based Access Control (RBAC)
13. Scope-Based / Fine-Grained Authorization
14. API Keys (Service-to-Service Auth)
15. Rate Limiting & Brute-Force Protection
16. Security Headers, CORS & Transport Security
17. Testing Secured Endpoints
18. OWASP API Security Top 10 — Applied Checklist
19. Production Hardening Checklist
20. Full Reference Project Structure

1. Core Concepts — Authentication vs Authorization

1.1 Authentication (AuthN) vs Authorization (AuthZ)

These two words are used interchangeably by beginners, but they are different layers of security and you implement them differently in FastAPI:

Concept	Question it answers	FastAPI mechanism
Authentication	Who are you?	OAuth2PasswordBearer + JWT verification
Authorization	What are you allowed to do?	Dependencies checking roles / scopes
Tokenization	How is identity carried between requests?	JWT (stateless) or opaque tokens (stateful)

A request to a secured FastAPI endpoint always goes through this pipeline:

```
Client Request
  ■
  ▼
[1] Extract token from Authorization header (Bearer <token>)
  ■
  ▼
[2] AUTHENTICATION: verify signature + expiry of JWT -> get user identity
  ■
  ▼
[3] AUTHORIZATION: check user's role/scopes against the endpoint's requirement
  ■
  ▼
[4] Route handler executes with a trusted `current_user` object
```

1.2 Stateless vs Stateful tokens

- **Stateless (JWT):** the server does not store the token. It trusts the signature. Fast, scales horizontally, but revoking a single token before expiry is hard.
- **Stateful (opaque / DB-backed):** the server stores a session or token record and checks the DB on every request. Slower, but instantly revocable.
- **Practical pattern used in production (and in this guide):** short-lived JWT *access token* (stateless, 10-15 min) + long-lived *refresh token* stored in DB (stateful, revocable, 7-30 days).

TIP: This hybrid model is what almost every real API (Google, GitHub, Stripe) uses. It gives you the speed of JWT for every normal request, and the revocation control of a database for the rare cases (logout, stolen token, password change).

2. Project Setup & Dependencies

2.1 Install dependencies

```
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

pip install fastapi uvicorn[standard]
pip install "python-jose[cryptography]" # JWT encode/decode
pip install "passlib[bcrypt]" # password hashing
pip install python-multipart # required for OAuth2 form login
pip install sqlmodel psycpg2-binary asyncpg # PostgreSQL ORM + driver
pip install pydantic-settings # settings/env management
pip install slowapi # rate limiting (Section 15)
```

TIP: python-jose is the standard JWT library used with FastAPI's own docs. PyJWT is a valid alternative — the concepts below apply to either.

2.2 Recommended project structure

```
app/
├── main.py # FastAPI() app, routers included here
├── core/
│   ├── config.py # Settings (SECRET_KEY, env vars)
│   ├── security.py # hashing + JWT create/verify functions
│   └── db/
│       ├── database.py # engine + session
│       ├── models.py # User, RefreshToken tables
│       └── schemas/
│           ├── auth.py # Pydantic request/response models
│           ├── deps.py # get_current_user, RoleChecker dependencies
│           └── routers/
│               ├── auth.py # /login /refresh /logout
│               └── users.py # protected user routes
```

2.3 Environment configuration (core/config.py)

```
# core/config.py
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    SECRET_KEY: str # 64+ random chars, load from .env
    ALGORITHM: str = "HS256"
    ACCESS_TOKEN_EXPIRE_MINUTES: int = 15
    REFRESH_TOKEN_EXPIRE_DAYS: int = 7
    DATABASE_URL: str

    class Config:
        env_file = ".env"

settings = Settings()
```

```
# .env
SECRET_KEY=8f3e9c1a7b2d4f6e8091a2b3c4d5e6f7091a2b3c4d5e6f7091a2b3c4d5e6f70
DATABASE_URL=postgresql+asyncpg://user:pass@localhost:5432/mydb
```

WARNING: Generate a strong SECRET_KEY with: `python -c "import secrets; print(secrets.token_hex(32))"`.
Never commit .env to git — add it to .gitignore.

3. Password Hashing

3.1 Why you never store plaintext or encrypt passwords

Passwords must be **hashed**, not encrypted. Encryption is reversible (bad — a leaked key exposes every password). Hashing is one-way: you can only verify a guess against the stored hash, never recover the original.

3.2 bcrypt with passlib

```
# core/security.py
from passlib.context import CryptContext

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(plain_password: str) -> str:
    return pwd_context.hash(plain_password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

3.3 Registration endpoint using the hash

```
@router.post("/register", status_code=201)
async def register(payload: UserCreate, session: AsyncSession = Depends(get_session)):
    existing = await session.execute(select(User).where(User.email == payload.email))
    if existing.scalar_one_or_none():
        raise HTTPException(status_code=400, detail="Email already registered")

    user = User(
        email=payload.email,
        hashed_password=hash_password(payload.password),
    )
    session.add(user)
    await session.commit()
    await session.refresh(user)
    return {"id": user.id, "email": user.email}
```

3.4 bcrypt vs argon2 — which to pick

Algorithm	Pros	When to use
bcrypt	Battle-tested, widely supported, simple	Default choice — fine for 99% of APIs
argon2id	Winner of Password Hashing Competition, tunable memory cost, strong resistance to GPU cracking	Higher security apps (e.g. fintech, admin panels)

```
# Using argon2 instead of bcrypt (just swap the scheme):  
pip install "passlib[argon2]"  
  
pwd_context = CryptContext(schemes=["argon2"], deprecated="auto")
```

WARNING: Never write your own hashing function and never use MD5/SHA1/SHA256 alone for passwords — they are too fast, making brute force trivial with GPUs.

4. Database Models (PostgreSQL via SQLAlchemy)

4.1 User model

```
# db/models.py
from sqlalchemy import SQLAlchemy, Field
from datetime import datetime
from typing import Optional
import uuid

class User(SQLAlchemy, table=True):
    id: uuid.UUID = Field(default_factory=uuid.uuid4, primary_key=True)
    email: str = Field(unique=True, index=True)
    hashed_password: str
    role: str = Field(default="user") # used later for RBAC (Section 12)
    is_active: bool = Field(default=True)
    created_at: datetime = Field(default_factory=datetime.utcnow)
```

4.2 RefreshToken model (needed for revocation — Sections 9-11)

```
class RefreshToken(SQLAlchemy, table=True):
    id: uuid.UUID = Field(default_factory=uuid.uuid4, primary_key=True)
    user_id: uuid.UUID = Field(foreign_key="user.id", index=True)
    token_hash: str = Field(index=True) # store a HASH of the token, never raw
    expires_at: datetime
    revoked: bool = Field(default=False)
    created_at: datetime = Field(default_factory=datetime.utcnow)
    replaced_by: Optional[uuid.UUID] = Field(default=None) # rotation chain (Section 9.3)
```

WARNING: Store a hash (e.g. SHA-256) of the refresh token in the database, exactly like a password. If your database leaks, raw refresh tokens should not be usable.

4.3 Async engine + session

```
# db/database.py
from sqlmodel import SQLModel
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession
from sqlalchemy.orm import sessionmaker
from core.config import settings

engine = create_async_engine(settings.DATABASE_URL, echo=False)
async_session = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

async def get_session() -> AsyncSession:
    async with async_session() as session:
        yield session

async def init_db():
    async with engine.begin() as conn:
        await conn.run_sync(SQLModel.metadata.create_all)
```

5. OAuth2 Password Flow in FastAPI

5.1 What FastAPI's OAuth2PasswordBearer actually does

It does NOT validate anything for you. It only defines **where** FastAPI should look for the token (the Authorization header) and powers the "Authorize" button in /docs. All real verification is code you write yourself (Section 6-7).

```
# core/security.py (continued)
from fastapi.security import OAuth2PasswordBearer

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="auth/login")
```

5.2 Login endpoint (issues access + refresh tokens)

```
# routers/auth.py
from fastapi import APIRouter, Depends, HTTPException, status
from fastapi.security import OAuth2PasswordRequestForm

router = APIRouter(prefix="/auth", tags=["auth"])

@router.post("/login")
async def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    session: AsyncSession = Depends(get_session),
):
    result = await session.execute(select(User).where(User.email == form_data.username))
    user = result.scalar_one_or_none()

    if not user or not verify_password(form_data.password, user.hashed_password):
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Incorrect email or password",
            headers={"WWW-Authenticate": "Bearer"},
        )

    access_token = create_access_token({"sub": str(user.id), "role": user.role})
    refresh_token = await create_refresh_token(user.id, session) # Section 10

    return {
        "access_token": access_token,
        "refresh_token": refresh_token,
        "token_type": "bearer",
    }
```

TIP: OAuth2PasswordRequestForm expects form-encoded fields `username` and `password` (not JSON) — this is the OAuth2 spec, not a FastAPI choice. It's why python-multipart must be installed.

7. Issuing Access Tokens

7.1 create_access_token()

```
# core/security.py (continued)
from jose import jwt
from datetime import datetime, timedelta, timezone
import uuid

def create_access_token(data: dict, expires_minutes: int | None = None) -> str:
    to_encode = data.copy()
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=expires_minutes or settings.ACCESS_TOKEN_EXPIRE_MINUTES
    )
    to_encode.update({
        "exp": expire,
        "iat": datetime.now(timezone.utc),
        "jti": str(uuid.uuid4()),
        "type": "access",
    })
    return jwt.encode(to_encode, settings.SECRET_KEY, algorithm=settings.ALGORITHM)
```

7.2 Decoding + verifying a token

```
from jose import JWTError, jwt

def decode_token(token: str) -> dict:
    try:
        payload = jwt.decode(token, settings.SECRET_KEY, algorithms=[settings.ALGORITHM])
        return payload
    except JWTError:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Could not validate credentials",
            headers={"WWW-Authenticate": "Bearer"},
        )
```

`jwt.decode()` automatically raises an error (caught above) if the signature is invalid OR if the token has expired — you don't need to check ``exp`` manually.

8. Protecting Routes with Dependencies

8.1 get_current_user dependency

```
# deps.py
from fastapi import Depends, HTTPException, status
from core.security import oauth2_scheme, decode_token
from db.database import get_session
from db.models import User
from sqlmodel import select

async def get_current_user(
    token: str = Depends(oauth2_scheme),
    session: AsyncSession = Depends(get_session),
) -> User:
    payload = decode_token(token)

    if payload.get("type") != "access":
        raise HTTPException(status_code=401, detail="Invalid token type")

    user_id = payload.get("sub")
    result = await session.execute(select(User).where(User.id == user_id))
    user = result.scalar_one_or_none()

    if user is None or not user.is_active:
        raise HTTPException(status_code=401, detail="User not found or inactive")

    return user
```

8.2 Using it in a route

```
# routers/users.py
@router.get("/me")
async def read_current_user(current_user: User = Depends(get_current_user)):
    return {"id": str(current_user.id), "email": current_user.email, "role":
current_user.role}
```

TIP: Every dependency in the chain runs BEFORE your route body. If the token is invalid, the request never reaches your handler — this is what makes FastAPI's Depends() model clean for security.

9. Refresh Tokens — Theory & Rotation Strategy

9.1 Why access tokens alone are not enough

If access tokens live for days, a stolen token stays valid for days — you can't revoke a stateless JWT before it expires. If you make access tokens live for 2 minutes, users get logged out constantly. Refresh tokens solve this tension:

Token	Lifetime	Stored where	Purpose
Access token	10-15 minutes	Client memory / header only	Authorizes each API call
Refresh token	7-30 days	DB (server) + httpOnly cookie or secure storage (direct)	Used only to mint new access tokens

9.2 The refresh flow

```
1. Client logs in -> gets access_token (15 min) + refresh_token (7 days)
2. Client calls API -> sends access_token in Authorization header
3. access_token expires (401) -> client calls POST /auth/refresh with refresh_token
4. Server verifies refresh_token against DB (not revoked, not expired)
5. Server issues a NEW access_token (and optionally a new refresh_token – rotation)
6. Client resumes calling the API with the new access_token
```

9.3 Refresh token rotation (advanced, recommended)

Every time a refresh token is used, issue a brand-new refresh token and immediately invalidate the old one. This limits the damage of a stolen refresh token to a single use.

- **Rotation:** old refresh token is marked revoked=True, new one is issued, linked via replaced_by.
- **Reuse detection:** if a revoked refresh token is presented again, this signals theft — immediately revoke the *entire token family* (every descendant token for that user) and force re-login.

NOTE: Reuse detection is what separates a beginner refresh-token system from a production-grade one. Without it, a stolen refresh token can be replayed silently for its entire lifetime.

10. Implementing Refresh Tokens (DB-backed, revocable)

10.1 Creating a refresh token

```
# core/security.py (continued)
import hashlib, secrets
from datetime import datetime, timedelta, timezone
from db.models import RefreshToken

def _hash_token(raw_token: str) -> str:
    return hashlib.sha256(raw_token.encode()).hexdigest()

async def create_refresh_token(user_id, session: AsyncSession, parent_id=None) -> str:
    raw_token = secrets.token_urlsafe(64) # random opaque string, NOT a JWT
    expires_at = datetime.now(timezone.utc) + timedelta(
        days=settings.REFRESH_TOKEN_EXPIRE_DAYS
    )

    record = RefreshToken(
        user_id=user_id,
        token_hash=_hash_token(raw_token),
        expires_at=expires_at,
    )
    session.add(record)
    await session.commit()

    return raw_token # return the RAW token to the client – only the hash is stored
```

TIP: Refresh tokens do not need to be JWTs. A long random string (`secrets.token_urlsafe`) is simpler and just as secure, since it's always validated against the database anyway — the DB lookup is what gives you revocation.

10.2 `/auth/refresh` endpoint with rotation + reuse detection

```

# routers/auth.py (continued)
from schemas.auth import RefreshRequest

@router.post("/refresh")
async def refresh_access_token(
    payload: RefreshRequest,
    session: AsyncSession = Depends(get_session),
):
    token_hash = _hash_token(payload.refresh_token)
    result = await session.execute(
        select(RefreshToken).where(RefreshToken.token_hash == token_hash)
    )
    stored = result.scalar_one_or_none()

    if stored is None:
        raise HTTPException(status_code=401, detail="Invalid refresh token")

    if stored.revoked:
        # REUSE DETECTED -> revoke every token for this user (possible theft)
        await session.execute(
            update(RefreshToken)
            .where(RefreshToken.user_id == stored.user_id)
            .values(revoked=True)
        )
        await session.commit()
        raise HTTPException(status_code=401, detail="Token reuse detected. Please log in again.")

    if stored.expires_at < datetime.now(timezone.utc):
        raise HTTPException(status_code=401, detail="Refresh token expired")

    # Rotate: revoke old, issue new
    stored.revoked = True
    new_raw_token = await create_refresh_token(stored.user_id, session)
    stored.replaced_by = new_raw_token # (store new token's id in production, simplified here)
    session.add(stored)
    await session.commit()

    result = await session.execute(select(User).where(User.id == stored.user_id))
    user = result.scalar_one()
    new_access_token = create_access_token({"sub": str(user.id), "role": user.role})

    return {
        "access_token": new_access_token,
        "refresh_token": new_raw_token,
        "token_type": "bearer",
    }

```

```

# schemas/auth.py
from pydantic import BaseModel

class RefreshRequest(BaseModel):
    refresh_token: str

```

11. Logout & Token Revocation / Blacklisting

11.1 Logout = revoke the refresh token

```
@router.post("/logout")
async def logout(
    payload: RefreshRequest,
    session: AsyncSession = Depends(get_session),
):
    token_hash = _hash_token(payload.refresh_token)
    result = await session.execute(
        select(RefreshToken).where(RefreshToken.token_hash == token_hash)
    )
    stored = result.scalar_one_or_none()
    if stored:
        stored.revoked = True
        session.add(stored)
        await session.commit()
    return {"detail": "Logged out"}
```

11.2 What about revoking an access token before it expires?

You can't invalidate a JWT itself (it's stateless), but you have two options if you need instant revocation (e.g. an admin force-disables an account):

Approach	How it works	Trade-off
Keep access tokens very short (10-15 min)	Damage window is naturally small	Simplest — the usual production answer
Redis-based deny-list of `jti`	On every request, check if token's jti is in a revoked set in Redis	Extra infra + a Redis call per request

TIP: For most APIs, short-lived access tokens + a DB-backed refresh token is sufficient. Only add a Redis deny-list if you have a hard requirement for instant kill-switch (e.g. compliance, banking).

12. Role-Based Access Control (RBAC)

12.1 Storing roles

The role field already exists on the User model (Section 4.1). Common values: user, editor, admin.

12.2 A reusable RoleChecker dependency

```
# deps.py (continued)
class RoleChecker:
    def __init__(self, allowed_roles: list[str]):
        self.allowed_roles = allowed_roles

    def __call__(self, current_user: User = Depends(get_current_user)) -> User:
        if current_user.role not in self.allowed_roles:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="You don't have permission to perform this action",
            )
        return current_user

allow_admin = RoleChecker(["admin"])
allow_editor_or_admin = RoleChecker(["editor", "admin"])
```

12.3 Using it

```
@router.delete("/users/{user_id}")
async def delete_user(user_id: str, current_user: User = Depends(allow_admin)):
    ... # only reachable by admins - 403 for everyone else
```

TIP: 401 Unauthorized = 'I don't know who you are' (bad/missing token). 403 Forbidden = 'I know who you are, but you're not allowed'. Don't mix these up — it matters for how clients handle the error (401 -> try refresh, 403 -> stop).

13. Scope-Based / Fine-Grained Authorization

13.1 When roles aren't granular enough

RBAC answers "what kind of user is this". Scopes answer "what specific actions has this particular token been granted" — useful for third-party API clients, service accounts, or fine-grained permissions like `orders:read`, `orders:write`.

13.2 Adding scopes with FastAPI's built-in support

```
from fastapi import Security
from fastapi.security import OAuth2PasswordBearer, SecurityScopes

oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl="auth/login",
    scopes={
        "orders:read": "Read order data",
        "orders:write": "Create or modify orders",
        "users:admin": "Manage users",
    },
)

async def get_current_user_with_scopes(
    security_scopes: SecurityScopes,
    token: str = Depends(oauth2_scheme),
    session: AsyncSession = Depends(get_session),
) -> User:
    payload = decode_token(token)
    token_scopes = payload.get("scopes", [])

    for scope in security_scopes.scopes:
        if scope not in token_scopes:
            raise HTTPException(
                status_code=403,
                detail=f"Missing required scope: {scope}",
                headers={"WWW-Authenticate": f'Bearer
scope="{security_scopes.scope_str}"'},
            )
    # ... resolve and return the user same as before
```

```
@router.post("/orders")
async def create_order(
    current_user: User = Security(get_current_user_with_scopes, scopes=["orders:write"]),
):
    ...

# When issuing the token at login, embed the scopes granted to that user/client:
access_token = create_access_token({"sub": str(user.id), "scopes": ["orders:read",
"orders:write"]})
```

14. API Keys (Service-to-Service Auth)

For machine-to-machine calls (webhooks, internal microservices, third-party integrations) API keys are often simpler than full OAuth2/JWT flows.

```
# deps.py
from fastapi.security import APIKeyHeader
from fastapi import Security

api_key_header = APIKeyHeader(name="X-API-Key")

async def verify_api_key(
    api_key: str = Security(api_key_header),
    session: AsyncSession = Depends(get_session),
) -> ServiceAccount:
    result = await session.execute(
        select(ServiceAccount).where(ServiceAccount.key_hash == _hash_token(api_key))
    )
    account = result.scalar_one_or_none()
    if not account or not account.is_active:
        raise HTTPException(status_code=401, detail="Invalid API key")
    return account
```

- Generate keys with `secrets.token_urlsafe(32)`, show the raw key to the user exactly once, store only the hash — identical principle to refresh tokens.
- Allow per-key scopes and per-key rate limits so a compromised key has limited blast radius.
- Support key rotation: let a user have two active keys during a transition window.

15. Rate Limiting & Brute-Force Protection

15.1 Why this belongs in an auth guide

JWTs and refresh tokens protect against forged identity — they do nothing against someone hammering /auth/login with a password list. You need rate limiting on auth endpoints specifically, not just globally.

15.2 slowapi (Flask-Limiter port for FastAPI)

```
pip install slowapi
```

```
# main.py
from slowapi import Limiter, _rate_limit_exceeded_handler
from slowapi.util import get_remote_address
from slowapi.errors import RateLimitExceeded

limiter = Limiter(key_func=get_remote_address)
app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler)
```

```
# routers/auth.py
@router.post("/login")
@limiter.limit("5/minute")
async def login(request: Request, form_data: OAuth2PasswordRequestForm = Depends(), ...):
    ...
```

15.3 Account lockout after repeated failures

- Track failed_login_attempts and locked_until on the User model.
- After N consecutive failures (e.g. 5), set locked_until = now + 15 minutes.
- Reset the counter to 0 on any successful login.
- Always return the same generic error ("Incorrect email or password") whether the email doesn't exist or the password is wrong — never reveal which one it was.

16. Security Headers, CORS & Transport Security

16.1 CORS — only allow the origins you actually trust

```
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://yourapp.com"], # never "*" if you send credentials/cookies
    allow_credentials=True,
    allow_methods=["GET", "POST", "PUT", "DELETE"],
    allow_headers=["Authorization", "Content-Type"],
)
```

16.2 Where to store tokens on the client

Storage	XSS risk	CSRF risk	Recommendation
localStorage	High — readable by any injected script	None	Avoid for tokens
httpOnly Secure cookie	Low — JS cannot read it	Needs CSRF token	Best for refresh tokens
In-memory (JS variable)	Low	None	Good for access tokens (lost on refresh, by de...

16.3 Essential HTTP security headers

```
# add via middleware or a reverse proxy (nginx/Caddy)
Strict-Transport-Security: max-age=63072000; includeSubDomains
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
Content-Security-Policy: default-src 'self'
Referrer-Policy: no-referrer-when-downgrade
```

WARNING: Always run production FastAPI behind HTTPS (terminate TLS at nginx/Caddy or your cloud load balancer). Sending JWTs over plain HTTP means anyone on the network can steal them.

17. Testing Secured Endpoints

17.1 pytest + httpx AsyncClient

```
pip install pytest pytest-asyncio httpx
```

```
# tests/test_auth.py
import pytest
from httpx import AsyncClient, ASGITransport
from main import app

@pytest.mark.asyncio
async def test_login_and_access_protected_route():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as client:
        # register
        await client.post("/auth/register", json={"email": "a@a.com", "password":
"Str0ngPass!"})

        # login
        resp = await client.post(
            "/auth/login",
            data={"username": "a@a.com", "password": "Str0ngPass!"},
        )
        assert resp.status_code == 200
        tokens = resp.json()

        # access protected route
        me = await client.get(
            "/users/me",
            headers={"Authorization": f"Bearer {tokens['access_token']}"},
        )
        assert me.status_code == 200
        assert me.json()["email"] == "a@a.com"
```

17.2 Testing that an expired/invalid token is rejected

```
@pytest.mark.asyncio
async def test_invalid_token_rejected():
    transport = ASGITransport(app=app)
    async with AsyncClient(transport=transport, base_url="http://test") as client:
        resp = await client.get("/users/me", headers={"Authorization": "Bearer
garbage.token.here"})
        assert resp.status_code == 401
```

18. OWASP API Security Top 10 — Applied Checklist

Risk	How this guide addresses it
API1: Broken Object Level Authorization	Always filter queries by <code>current_user.id</code> — never trust an id from the URL/body alone (S
API2: Broken Authentication	Strong hashing, short-lived JWTs, rate-limited login, account lockout (Sec. 3, 7, 15)
API3: Broken Object Property Level Authorization	Use explicit response models (Pydantic) — never return full ORM objects with hidden fi
API4: Unrestricted Resource Consumption	Rate limiting on all endpoints, pagination limits, request size limits (Sec. 15)
API5: Broken Function Level Authorization	RoleChecker / scope dependencies on every sensitive route (Sec. 12, 13)
API6: Unrestricted Access to Sensitive Business Flows	Rate-limit and log sensitive flows (password reset, payments) separately
API7: Server Side Request Forgery	Validate/allowlist any user-supplied URL before the server fetches it (not covered by au
API8: Security Misconfiguration	Restrict CORS, set security headers, disable /docs in production if desired (Sec. 16)
API9: Improper Inventory Management	Version your API (/v1, /v2), retire old versions, document all endpoints
API10: Unsafe Consumption of APIs	Validate and sanitize data coming back from third-party APIs you call

19. Production Hardening Checklist

- 1 SECRET_KEY is 256-bit random, stored in env vars / secrets manager — never in code.
- 2 Access tokens expire in 10-15 minutes; refresh tokens in 7-30 days.
- 3 Refresh tokens are stored as hashes in the DB, with rotation + reuse detection enabled.
- 4 Passwords hashed with bcrypt or argon2id, never MD5/SHA/plaintext.
- 5 Login, register, and refresh endpoints are rate-limited.
- 6 HTTPS enforced everywhere; HSTS header set.
- 7 CORS allow_origins is an explicit list, never "*" when allow_credentials=True.
- 8 Every protected route uses a dependency — no manual token checks scattered in route bodies.
- 9 Sensitive routes checked for object-level authorization (user can only touch their own data).
- 10 Structured logging on auth events (login success/fail, token refresh, logout) for audit trails.
- 11 Automated tests cover: valid login, invalid login, expired token, wrong role, token reuse.
- 12 Dependabot/pip-audit or similar running to catch vulnerable dependency versions.

20. Full Reference Project Structure

Putting every section together, a complete secured FastAPI project looks like this:

```
app/
├── main.py
├── core/
│   ├── config.py # Section 2.3
│   ├── security.py # Sections 3, 6, 7, 10 (hashing + JWT + refresh helpers)
│   └── db/
│       ├── database.py # Section 4.3
│       ├── models.py # Section 4.1, 4.2 (User, RefreshToken, ServiceAccount)
│       └── schemas/
│           ├── auth.py # RefreshRequest, TokenResponse, UserCreate, UserRead
│           ├── deps.py # get_current_user, RoleChecker, verify_api_key (Sec 8, 12, 14)
│           └── routers/
│               ├── auth.py # /register /login /refresh /logout (Sec 5, 10, 11)
│               └── users.py # /me, admin-only routes (Sec 8, 12)
└── tests/
    └── test_auth.py # Section 17
```

```
# main.py – wiring it all together
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from db.database import init_db
from routers import auth, users

app = FastAPI(title="Secure API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://yourapp.com"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.on_event("startup")
async def on_startup():
    await init_db()

app.include_router(auth.router)
app.include_router(users.router)
```

Recommended learning path from here: implement Sections 2-8 first (basic login + protected routes) and get it fully working end-to-end. Then add Sections 9-11 (refresh tokens with rotation). Only after that add RBAC/scopes (12-13), then harden with rate limiting and headers (15-16) before deploying.